The Word RAM in Lean

1 The Word RAM in Lean

This document explains the word RAM defined in the Lean file `Abstract.lean`, so that a reader can check its definitions. The boxes quote it and `AbstractBridge.lean` at commit `4e85a95c`; both files are in `Complexity/Computability/WordRAM/`.

1.1 Imports

`Abstract.lean` imports only Mathlib and CSLib's base module `Cslib.Init`, and defines the machine from scratch. (The source file remains the authoritative version.)

Lean

```
public import Cslib.Init
public import Mathlib.Data.Nat.Notation
public import Mathlib.Data.Set.Defs
public import Mathlib.Logic.Function.Basic
```

1.2 Words and Operations

At word size w, a *word* is a natural number below 2^w. The machine stores arbitrary natural numbers, but an instruction faults when it would create a value that is undefined or not a word (Section 1.4).

There are ten binary operations, and each computes its exact value. Subtraction $x - y$ is undefined for $y > x$, and division x/y and remainder x mod y are undefined for $y = 0$; otherwise division rounds down. The operations `and`, `or` and `xor` act bitwise, and `shl` and `shr` shift x left and right by y bits, that is, give $x \cdot 2^y$ and $\lfloor x/2^y \rfloor$. `AluOp.eval` gives the value of an operation on natural numbers, before any check that it is a word, or none where it is undefined.

Lean

```
inductive AluOp
  | add | sub | mul | div | mod | and | or | xor | shl | shr

def AluOp.eval : AluOp → ℕ → ℕ → Option ℕ
  | .add, x, y => some (x + y)
  | .sub, x, y => if y ≤ x then some (x - y) else none
  | .mul, x, y => some (x * y)
  | .div, x, y => if y = 0 then none else some (x / y)
  | .mod, x, y => if y = 0 then none else some (x % y)
  | .and, x, y => some (x &&& y)
  | .or, x, y => some (x ||| y)
  | .xor, x, y => some (x ^^^ y)
  | .shl, x, y => some (x * 2 ^ y)
  | .shr, x, y => some (x / 2 ^ y)
```

1.3 Instructions and Programs

A machine has r registers $R[0], \ldots, R[r-1]$ and a memory cell $M[a]$ at every address $a \in \mathbb{N}$. The number r of registers is a parameter of the types below (the implicit variable `r`). There are ten instructions; the comment on each gives its effect. Register arguments d, i, j, a range over `Fin r` $= \{0, \ldots, r-1\}$ (in `load` and `store`, register a holds the address); a constant x and a jump target t are arbitrary natural numbers. The instruction `width` reads the word size w, so a program can depend on it. A *program* is a finite list of instructions, numbered from 0.

```lean
variable {r : ℕ}

inductive Instr (r : ℕ)
  | const (d : Fin r) (x : ℕ)         -- `R[d] := x`
  | copy (d i : Fin r)                -- `R[d] := R[i]`
  | alu (f : AluOp) (d i j : Fin r)   -- `R[d] := f(R[i], R[j])`
  | width (d : Fin r)                 -- `R[d] := w`, the word size
  | load (d a : Fin r)                -- `R[d] := M[R[a]]`
  | store (a i : Fin r)               -- `M[R[a]] := R[i]`
  | jump (t : ℕ)                      -- go to instruction `t`
  | branchEq (i j : Fin r) (t : ℕ)    -- go to `t` if `R[i] = R[j]`, else to the next one
  | branchLt (i j : Fin r) (t : ℕ)    -- go to `t` if `R[i] < R[j]`, else to the next one
  | halt (i : Fin r)                  -- stop with output `R[i]`

abbrev Program (r : ℕ) := List (Instr r)
```

1.4 States and One Step

A *state* consists of the number pc of the next instruction, the registers and the memory. The memory is a function on all of $\mathbb{N}$: there is no allocation and no bound on addresses.

One step of a program continues in a new state, halts with an output, or faults. It executes instruction number pc:

- `const` and `alu` write their value to $R[d]$ if it is a word, and fault otherwise. There is no wrap-around modulo 2^w. An `alu` whose value is undefined (none) also faults.
- `width` writes w, which is always a word, since $w < 2^w$.
- `copy`, `load` and `store` move a value without a check.
- `jump` and the two branches set pc; every other instruction except `halt` moves on to instruction pc + 1.
- `halt` i stops with output $R[i]$.
- If pc is not the number of an instruction, that is, pc is at least the length of the program, the step faults.

Copies, loads and stores need no check: when the input fits (Section 1.6), an induction along the run shows that every register and memory cell always holds a word, and so every address used is below 2^w. (This induction is not stated in the abstract files; it only explains why the check is not needed. The library states it for its own word RAM as `Cslib.WordRAM.Runs.bounded_init`.)

```lean
structure State (r : ℕ) where
  pc : ℕ
  reg : Fin r → ℕ
  mem : ℕ → ℕ

inductive Outcome (r : ℕ)
  | next (s : State r)
  | halt (v : ℕ)
  | fault

def step (w : ℕ) (p : Program r) (s : State r) : Outcome r :=
  let set (d : Fin r) (x : ℕ) : Outcome r :=  -- `R[d] := x`, then the next instruction
    .next { s with pc := s.pc + 1, reg := Function.update s.reg d x }
  let setWord (d : Fin r) (x : ℕ) : Outcome r := if x < 2 ^ w then set d x else .fault
  match p[s.pc]? with
  | none => .fault
  | some (.const d x) => setWord d x
  | some (.copy d i) => set d (s.reg i)
  | some (.alu f d i j) =>
    match f.eval (s.reg i) (s.reg j) with
    | some x => setWord d x
    | none => .fault
  | some (.width d) => set d w  -- `w < 2 ^ w`, so this never faults
  | some (.load d a) => set d (s.mem (s.reg a))
  | some (.store a i) =>
    .next { s with pc := s.pc + 1, mem := Function.update s.mem (s.reg a) (s.reg i) }
  | some (.jump t) => .next { s with pc := t }
  | some (.branchEq i j t) => .next { s with pc := if s.reg i = s.reg j then t else s.pc + 1 }
  | some (.branchLt i j t) => .next { s with pc := if s.reg i < s.reg j then t else s.pc + 1 }
  | some (.halt i) => .halt (s.reg i)
```

1.5 Running Time

`Halts w p s v n` says that from state s, at word size w, program p halts with output v after exactly n instructions, the final `halt` included. It follows `step` one instruction at a time: a fault ends the run without halting, and a run that never halts satisfies `Halts` for no v and n. Every instruction takes one unit of time, whatever its operands and the word size, so n is the running time.

```lean
def Halts (w : ℕ) (p : Program r) : State r → ℕ → ℕ → Prop
  | _, _, 0 => False
  | s, v, n + 1 =>
    match step w p s with
    | .next t => Halts w p t v n
    | .halt x => x = v ∧ n = 0
    | .fault => False
```

1.6 Input

An input is a list $x_1, \ldots, x_k$ of natural numbers. The start state is at instruction 0, has all registers 0, and has $k, x_1, \ldots, x_k$ in memory at addresses $0, 1, \ldots, k$ and 0 at every other address; so a program reads the length

of its input at $M[0]$. The input *fits* word size w if its length k and all its entries are words. The output of a run is the single number $R[i]$ at its `halt` i.

```lean
def init (input : List ℕ) : State r :=
  ⟨0, fun _ => 0, fun a => (input.length :: input).getD a 0⟩

def InputFits (w : ℕ) (input : List ℕ) : Prop :=
  input.length < 2 ^ w ∧ ∀ x ∈ input, x < 2 ^ w
```

1.7 Decision in Time T

A decision problem is a set $A \subseteq \alpha$ of instances, given to the machine by an encoding *encode*: $\alpha \to$ `List` $\mathbb{N}$. Two functions on instances set the terms: $T(a)$ bounds the running time on instance a, and $bits(a)$ is the word-size threshold: the program must work at every word size from $bits(a)$ on that the input fits. A program p *decides* A *in time* T if, for every instance a and every word size $w \geq bits(a)$ that $encode(a)$ fits, p halts within $T(a)$ instructions, with output 1 if $a \in A$ and 0 if not. One program serves every such word size, and can read it with `width`. Word sizes that the input does not fit impose nothing, so the input need not fit at $w = bits(a)$ itself. A is *decidable in time* T if some program, with some number of registers, decides it. (The length condition gives $k < 2^w$, so all input addresses $0, \ldots, k$ fit in a word; for $k > 0$ this is the same as $w > \log_2 k$. `bits` can ask for more.)

```lean
def Decides {α : Type*} (p : Program r) (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) :
    Prop :=
  ∀ a w, bits a ≤ w → InputFits w (encode a) →
    ∃ v n, Halts w p (init (encode a)) v n ∧ n ≤ T a ∧ (a ∈ A → v = 1) ∧ (a ∉ A → v = 0)

def DecidableInTime {α : Type*} (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) : Prop :=
  ∃ r, ∃ p : Program r, Decides p A encode T bits
```

1.8 Main Theorems

Two theorems say that the machine of this section is the library's word RAM `Cslib.WordRAM` over its exact basis `.exact`. They are stated in `AbstractBridge.lean`, which imports both `Abstract.lean` and the library's word RAM, in the namespace `Cslib.WordRAM.AbstractBridge`. There, names with the prefix `Abstract.` are those of this section, and names without it are from the library's word RAM `Cslib.WordRAM`: `Program .exact r` is a program over the exact basis with r registers, a state `⟨pc, ⟨reg, mem⟩⟩` has the same three parts as here, `step` is one step, and `Executes w q s v n` says that q halts from s with output v after exactly n instructions. A reader does not need to know how these library definitions are made: `decidableInTime_iff` makes the library's decidability in time T equal to that of this section, and `runsAgree` does the same for single steps and halting runs.

Two outcomes, one of each machine, *agree* if both continue in the same state, both halt with the same output, or both fault. An abstract program p and a library program q *run alike* if, from every state and at every word size, one step of each gives agreeing outcomes, and p halts with output v after n instructions exactly when q does.

```lean
def OutcomeAgrees : Abstract.Outcome r → Outcome r → Prop
  | .next s, .next t => t = ⟨s.pc, ⟨s.reg, s.mem⟩⟩
  | .halt v, .halt v' => v = v'
  | .fault, .fault => True
  | _, _ => False

def Same (p : Abstract.Program r) (q : Program .exact r) : Prop :=
  ∀ w pc reg mem,
    OutcomeAgrees (Abstract.step w p ⟨pc, reg, mem⟩) (step w q ⟨pc, ⟨reg, mem⟩⟩) ∧
    ∀ v n, Abstract.Halts w p ⟨pc, reg, mem⟩ v n ↔ Executes w q ⟨pc, ⟨reg, mem⟩⟩ v n
```

For every number r of registers, every abstract program runs alike with some library program, and every library program runs alike with some abstract program. (In the proof, the program of the other machine is a translation instruction by instruction.)

```lean
theorem runsAgree (r : ℕ) :
    (∀ p : Abstract.Program r, ∃ q : Program .exact r, Same p q) ∧
      ∀ q : Program .exact r, ∃ p : Abstract.Program r, Same p q := ...
```

The two machines have the same time classes: for every type α of instances, problem A, encoding, time bound T and word-size threshold *bits*, A is decidable in time T on the machine of this section exactly when it is on the library's word RAM over the exact basis.

```lean
theorem decidableInTime_iff {α : Type u} (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) :
    Abstract.DecidableInTime A encode T bits ↔
      DecidableInTime .exact A encode T bits := ...
```

So, by `decidableInTime_iff`, a library theorem that concludes `DecidableInTime .exact A encode T bits` is a time bound on the machine described in this section.

To trust these results, a reader needs to read only the definitions of `Abstract.lean` and the statements of these two theorems, with `OutcomeAgrees` and `Same`. Lean checks the proofs, which use only Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`).